🌐 Services & Service Discovery

Intermediate Kubernetes — Module 4 of 13

Enterprise EKS Platform

📋 Module Agenda

Core Concepts

- Why Services matter
- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName
- Selectors, Endpoints, and EndpointSlices

Advanced Topics

- DNS-based and environment variable discovery
- kube-proxy modes and traffic policies
- Headless Services and multi-port patterns
- AWS Load Balancer Controller

🔌 Why Services Matter

Stable networking over ephemeral Pods

The Pod Networking Problem

Pod IPs are ephemeral. Every time a Pod restarts, it receives a new IP address from the CNI plugin.

- Pods are mortal — Deployments replace them on failure or scaling events
- ReplicaSets create multiple Pods for the same workload — which IP do clients use?
- Rolling updates replace Pods one by one with new IPs
- Nodes can fail, causing Pods to reschedule on different nodes
- Hard-coding IPs is fragile and defeats the purpose of orchestration

Key insight: You need a stable endpoint that abstracts over a dynamic set of Pods.

The Service Abstraction

What a Service Provides

- **Stable virtual IP** (ClusterIP) that never changes
- **Stable DNS name** that resolves to the virtual IP
- **Load balancing** across healthy backend Pods
- **Automatic endpoint updates** as Pods come and go
- **Decoupling** between producers and consumers

Result: Clients connect to a Service name or IP — Kubernetes handles routing to the right Pod.

ClusterIP Service

Default Service Type

- Allocates a virtual IP from the Service CIDR
- Accessible **only within** the cluster
- kube-proxy programs iptables/IPVS rules
- Round-robin load balancing by default
- Most common type for internal communication

NodePort Service

Extends ClusterIP

- Opens a static port (30000–32767) on every node
- Still creates a ClusterIP internally
- External traffic: `<NodeIP>:<NodePort>`
- kube-proxy routes to backend Pods
- Port range configurable via API server flag

LoadBalancer Service

Cloud-Integrated Load Balancing

- Extends NodePort — provisions an external load balancer via cloud provider
- On AWS EKS: creates a Classic LB, NLB, or ALB depending on annotations
- Assigns a public or internal DNS name to the load balancer
- Each LoadBalancer Service gets its own cloud LB (cost consideration)

ExternalName Service

DNS Alias (CNAME)

- Maps a Service name to an external DNS name
- Returns a CNAME record, **not** a ClusterIP
- No selectors or endpoints
- No proxying — DNS resolution only
- Useful for referencing external databases or APIs

📈 Service Type Comparison

Type	Scope	Access Method	Use Case
ClusterIP	Internal only	Virtual IP / DNS	Inter-service communication
NodePort	Internal + external	NodeIP:Port	Development, bare-metal
LoadBalancer	Internal + external	Cloud LB DNS	External access via cloud LB
ExternalName	DNS alias	CNAME record	External service abstraction

Remember: Each type builds on the previous — LoadBalancer ⊃ NodePort ⊃ ClusterIP.

Selectors & Label Matching

Matching Rules

- Selector uses **equality-based** matching
- Pod must have **all** selector labels
- Pod can have **additional** labels
- Labels must match in the **same namespace**
- Services are namespace-scoped

Endpoints & EndpointSlices

Endpoints (Legacy)

- Single object per Service listing all Pod IPs
- Scales poorly beyond ~1000 endpoints
- Any Pod change rewrites the entire object

EndpointSlices (Modern)

- Splits endpoints into chunks (max 100 per slice)
- More efficient updates at scale
- Default since Kubernetes 1.21
- Supports dual-stack (IPv4 + IPv6)

🔍 Service Discovery

How applications find each other in the cluster

DNS-Based Discovery (CoreDNS)

How It Works

- CoreDNS runs as a Deployment in `kube-system` namespace
- Every Service gets a DNS record automatically
- Pods are configured to use CoreDNS via `/etc/resolv.conf`
- CoreDNS watches the Kubernetes API for Service/Endpoint changes
- Supports A, AAAA, SRV, and CNAME records

In practice: Applications simply connect to the Service name — DNS handles the rest.

DNS Record Formats

Fully Qualified Domain Names

- **Service A record:** `<service>.<namespace>.svc.cluster.local`
- **SRV records** for port discovery: `_http._tcp.<service>...`
- **Pod DNS** (hostname/subdomain set): `<pod>.<service>.<namespace>.svc...`

> **Shorthand works within the same namespace:** Just use `backend-api` instead of the full FQDN. Cross-namespace: use `backend-api.production`.

Environment Variable Discovery

- kubelet injects env vars for each active Service
- Format: `{SVC_NAME}_SERVICE_HOST` and `_SERVICE_PORT`
- Only Services that exist **before** the Pod starts
- Same namespace only

Limitation: Order-dependent — Service must exist before the Pod. DNS has no such constraint.

Headless Services

clusterIP: None

- No virtual IP allocated
- DNS returns **all Pod IPs** directly
- Client-side service discovery
- Essential for StatefulSets
- Used by databases, message queues, ZooKeeper

📦 kube-proxy & Traffic Flow

Understanding the data plane

kube-proxy Modes

iptables Mode (Default)

- Programs iptables NAT rules
- Random backend selection
- O(n) rule evaluation
- Proven, widely deployed
- Performance degrades at >5000 Services

IPVS Mode

- Uses Linux IPVS (kernel-level L4 LB)
- O(1) connection routing with hash tables
- Multiple algorithms: rr, lc, sh, dh
- Better performance at scale
- Requires IPVS kernel modules

EKS default: iptables mode. Consider IPVS for clusters with thousands of Services.

Traffic Policies

Cluster (Default)

- Traffic can route to **any Pod** in the cluster
- Even load distribution
- Extra network hop if Pod is on another node
- Source IP is SNATed (masked)

Local

- Traffic only routes to Pods on the **same node**
- Preserves source IP
- Avoids extra hop (lower latency)
- Risk of uneven distribution

Session Affinity

- Routes requests from the same client IP to the same Pod
- Configured via `sessionAffinity: ClientIP`
- Default timeout: 10800s (3 hours)
- Only supports ClientIP — no cookie-based affinity at L4

Best practice: Design stateless apps. Use affinity only for legacy requirements.

🚀 Advanced Service Patterns

Recommended configurations and patterns

Multi-Port Services

- Each port **must** have a unique `name`
- `targetPort` can reference a named port on the Pod
- SRV records created for each named port

Named ports decouple the Service from specific port numbers — Pods can change ports independently.

ExternalTrafficPolicy Deep Dive

Cluster vs Local — Trade-offs in Practice

Aspect	Cluster	Local
Load Distribution	Even across all Pods	Uneven if Pods ≠ nodes
Source IP	SNATed (lost)	Preserved
Health Checks	N/A	LB health checks node:healthCheckNodePort
Network Hops	Possible extra hop	No extra hop
Failure Mode	Any Pod can serve	Nodes without Pods get no traffic

Recommendation: Use `externalTrafficPolicy: Local` when source IP preservation is required for security logging or geo-routing.

Topology-Aware Routing

Keep Traffic Local

- Prefers routing to Pods in the same AZ
- Reduces cross-AZ data transfer costs
- Lower latency for east-west traffic
- Falls back to other zones if no local Pods

Service Mesh Concepts

What It Adds

- mTLS encryption between services
- Traffic management, canaries, and circuit breaking

Istio

- Feature-rich Envoy sidecar (or ambient mode)
- VirtualService / DestinationRule CRDs

Linkerd

- Lightweight Rust-based proxy
- Lower overhead, graduated CNCF project

When to adopt: Consider a service mesh when you need mTLS between all services, advanced traffic splitting, or deep observability across microservices.

☁️ AWS Load Balancer Controller

AWS Load Balancer Integration

- Replaces the in-tree cloud provider for LB provisioning
- Creates **NLB** for LoadBalancer Services, **ALB** for Ingress resources
- Supports IP-mode targets (bypasses NodePort, routes directly to Pods)
- Integrates with AWS WAF, Shield, and ACM for TLS

NLB Target Modes on EKS

Instance Mode

- NLB targets = EC2 instances
- Traffic: NLB → NodePort → kube-proxy → Pod
- Extra network hop through NodePort
- Works with any CNI

IP Mode (Recommended)

- NLB targets = Pod IPs directly
- Traffic: NLB → Pod (one hop)
- Lower latency, better performance
- Requires VPC CNI plugin

EKS best practice: Use IP mode with the VPC CNI plugin for optimal performance and source IP preservation.

🎯 Service Best Practices

✅ Do

- Use DNS for service discovery
- Use named ports in targetPort
- Set `externalTrafficPolicy: Local` when source IP matters
- Enable topology-aware routing in multi-AZ clusters
- Use IP-mode NLB on EKS
- Always define readiness probes

❌ Avoid

- Hard-coding Pod IPs anywhere
- Relying on env vars for discovery
- Creating one LoadBalancer per service
- Ignoring empty endpoints warnings
- Using session affinity for new applications
- Skipping readiness probes (sends traffic to unready Pods)

🧑‍💻 Lab 4: Services and Discovery

Hands-On Exercise

- Deploy a multi-tier app and create ClusterIP and NodePort Services
- Test DNS-based service discovery and examine DNS records
- Explore Endpoints and EndpointSlices as pods scale
- Deploy a headless Service and compare DNS responses
- Connect the frontend to the backend over Service DNS
- *Optional:* Provision a LoadBalancer Service and verify external access

Duration: ~30 minutes

🎯 Key Takeaways

1. **Services are the networking primitive** — stable endpoints over dynamic Pods. Never hard-code Pod IPs.
2. **DNS is the standard discovery mechanism** — use short names within the namespace, FQDNs across namespaces. Headless Services return Pod IPs directly.
3. **Choose the right traffic policy** — `Local` for source IP preservation, `Cluster` for even distribution.
4. **Use AWS Load Balancer Controller on EKS** — IP-mode NLB targets give direct Pod routing with lower latency.

❓ Questions & Discussion

Module 4: Services & Service Discovery

Up Next: Module 5 — Managing Application Configuration and Secrets